

Campus Assistant (LLM + RAG + Multi Agent LangGraph + MCP)

1) Goal

Build a campus assistant that can answer student questions with citations, check timetables and deadlines using tools, and generate study plans. MCP is the standard connector layer for tools and data sources, so your agent talks to campus systems through MCP servers, not direct one-off integrations. ([Anthropic](#))

2) What you will build

A web app with:

- Student chat interface (streaming responses)
- Admin interface for uploading campus documents and timetable CSV
- RAG pipeline for document grounded answers with citations
- Multi-agent orchestration using LangGraph (supervisor plus specialist agents)
- MCP servers that expose tools and resources (docs, timetable, notices) to the agent ([LangChain](#))

3) Core features (MVP)

Student side

- Ask policy questions, for example attendance rules, exam procedure, hostel timings
- Ask timetable questions, for example “What classes do I have Friday”
- Ask deadline questions, for example “When is the CS101 assignment due”
- Study plan generator, based on subjects, exam date, hours per day
- Answers include citations (doc name, page, section)

Admin side

- Upload PDFs and DOCX (stored locally)
- Tag documents (department, year, course)
- Upload timetable CSV (stored locally)
- Rebuild index (re chunk, re embed, update vector store)
- View logs (queries, latency, tool calls, top sources)

4) Architecture (clear and grade friendly)

A) Frontend

- Next.js or React
- Chat UI with streaming
- Admin upload and dashboard pages

B) Backend API (your app server)

- FastAPI (Python) or Express (Node)
- Auth with simple JWT (student, admin)
- Chat endpoint that runs LangGraph orchestration
- Document ingestion endpoints
- Local storage folder for docs and indexes

C) Local data layer

- Local file storage: `./data/docs`, `./data/timetable`, `./data/index`
- Vector store: Qdrant (local Docker) or FAISS (local file)
- SQLite or Postgres (optional) for metadata, users, logs

D) MCP layer (important part)

Run 2 to 3 MCP servers locally. Each MCP server exposes tools, resources, and optionally prompts that your app can discover and call in a standard way. ([Model Context Protocol](#))

Recommended MCP servers:

1. Docs MCP Server

- Tools:
 - `docs.search(query, filters)`
 - `docs.get_chunk(doc_id, chunk_id)`
- Resources:
 - list of available docs, metadata, tags

2. Timetable MCP Server

- Tools:
 - `timetable.get(day, student_group)`
 - `deadlines.get(course_id)`
- Resources:
 - timetable CSV versions, last updated

3. Notices MCP Server (optional)

- Tools:
 - `notices.latest(department, limit)`
- Resources:
 - notice feed items

Your backend is the MCP client; it calls MCP tools when the LangGraph agents decide they are needed.

5) LangGraph multi-agent design (main highlight)

Use LangGraph as the orchestrator. Each agent is a node, edges decide who runs next, and a shared state holds the conversation, retrieved context, and tool results. ([LangChain Docs](#))

Agents (nodes)

1. **Supervisor Agent**
 - Reads the user query
 - Routes to the right specialist agent
 - Enforces rules, for example citations required for policy answers
2. **Retriever Agent (RAG)**
 - Calls Docs MCP tools to search and fetch relevant chunks
 - Builds a context pack with citations
3. **Timetable Agent**
 - Calls Timetable MCP tools
 - Formats schedule output cleanly
4. **Planner Agent**
 - Generates study plans
 - Uses deadlines tool if needed
 - Produces structured output (day wise plan)
5. **Response Writer Agent**
 - Produces final answer using the context pack and tool outputs
 - Always includes citations when docs were used
6. **Safety and Scope Agent (Optional)**
 - If retrieval returns nothing, forces “I do not have this in the uploaded docs” behavior
 - Blocks unsafe requests if any

Shared state (example fields)

- messages
- user_profile (course, department, semester)
- intent
- retrieved_chunks (with citations)
- tool_results
- final_answer

6) Tool calling flow (how it works in code)

- Your backend sends the model the tool schemas (MCP tool wrappers)
- Model requests a tool call
- Your backend executes the call (via MCP client)
- Tool result is fed back into the graph state

- Final response is generated

This aligns with standard function and tool calling patterns. ([OpenAI Platform](#))

7) RAG pipeline details (local)

1. **Ingestion**
 - Parse PDF or DOCX to text per page
 - Store metadata: doc_id, title, tags, page number
2. **Chunking**
 - 400 to 800 token chunks
 - 80 to 120 overlap
 - Preserve headings if possible
3. **Embeddings and indexing**
 - Generate embeddings for chunks
 - Store in vector store with metadata
4. **Retrieval**
 - Semantic search top k
 - Return chunks with citations (doc title, page, section)
 - If top k is empty, answer must state missing data

8) Evaluation and observability (big differentiator)

Offline evaluation set

Create 50 to 80 campus questions with expected answer and expected source doc.

Track:

- correctness (manual 1 to 5)
- citation correctness (right doc and page)
- latency
- tool call success rate

Runtime logs

Log per query:

- total latency
- token usage
- which MCP server tools were called
- which docs were cited

9) Suggested tech stack (simple)

- Frontend: Next.js + Tailwind
- Backend: FastAPI
- LangGraph: LangChain LangGraph

- Vector store: Qdrant local Docker or FAISS local
- Storage: local folders
- MCP: local MCP servers for docs, timetable, notices ([LangChain](#))

10) Implementation plan (6 weeks)

- **Week 1:** UI chat, auth, backend chat endpoint; Admin upload UI and local storage
- **Week 2:** Ingestion, chunking, embeddings, vector store; Basic RAG answers with citations
- **Week 3:** MCP servers for docs and timetable; Backend MCP client wrapper functions
- **Week 4:** LangGraph orchestration with supervisor plus 3 specialist agents; Tool calling integrated into graph
- **Week 5:** Admin rebuild index, filters by department and course; Logging and evaluation dataset runner
- **Week 6:** Polish outputs, add notices MCP server (optional); Final report, demo script, screenshots

11) Deliverables for college submission

- Source code repository
- Project report:
 - architecture diagram
 - LangGraph workflow description
 - MCP servers and tools list
 - RAG pipeline with citations logic
 - evaluation results and logs
- Demo video (5 to 8 minutes)